

威胁情报驱动的多智能体 Android 反运行时分析方法^{*}

严欣芃¹, 张向宇^{1,4}, 范玲玲^{1,4}, 苏煜程³, 陈森¹, 刘哲理¹, 操晓春^{2,4}



¹(南开大学 密码与网络空间安全学院, 天津 300350)

²(中山大学 网络空间安全学院, 广东 深圳 518107)

³(小红书, 上海 200031)

⁴(北京中关村学院, 北京 100094)

通信作者: 范玲玲, E-mail: linglingfan@nankai.edu.cn

摘要: Android 生态系统的开放性与碎片化使其面临着持续演进且日益复杂的安全威胁, 越来越多的恶意攻击者使用运行时分析技术来窥探应用逻辑、篡改执行流程, 从而规避 Android 安全措施, 加强恶意行为的持久性与影响范围. 反运行时分析 (anti-runtime analysis) 技术通过检测运行时分析行为, 使用户与开发者能够及时发现并采取防护措施, 保障应用完整性与数据安全. 目前, 研究人员已提出多种基于静态数据库的反运行时分析方法, 即收集已知运行时分析技术的特征情报, 将从设备中提取的信息与其进行匹配, 从而发现运行时分析技术的痕迹. 但这类方法存在“知识滞后”问题, 且分析深度和结果可解释性不足, 使其在高速迭代的运行时分析技术面前愈发乏力. 为此, 提出并实现一个基于威胁情报驱动的多智能体 Android 反运行时分析方法——AIntel-Agent, 该方法利用智能体技术构建动态知识库, 从 Root、Hook、调试、虚拟环境、篡改这 5 类运行时分析技术的特征出发对设备指纹进行深度分析, 自动化、智能化地完成 Android 反运行时分析任务. AIntel-Agent 通过智能体协作框架的设计, 提出一种从互联网实时获取威胁情报构建动态知识库的方法, 并构建循环推理机制使智能体将设备指纹与运行时分析技术的特征关联, 形成设备反运行时分析报告. 基于现实设备数据集的实验结果表明, AIntel-Agent 在 Android 设备达到 88.5% 的精确率与 93.9% 的召回率, 其风险报告具备充足证据与可解释性, 证明了其在 Android 反运行时分析中的有效性.

关键词: Android 反运行时分析; 大语言模型; 多智能体系统; 软件分析; 移动安全

中图法分类号: TP309

中文引用格式: 严欣芃, 张向宇, 范玲玲, 苏煜程, 陈森, 刘哲理, 操晓春. 威胁情报驱动的多智能体 Android 反运行时分析方法. 软件学报. <http://www.jos.org.cn/1000-9825/7726.htm>

英文引用格式: Yan XP, Zhang XY, Fan LL, Su YC, Chen S, Liu ZL, Cao XC. Threat-intelligence-driven Multi-agent Android Anti-runtime Analysis Method. Ruan Jian Xue Bao/Journal of Software (in Chinese). <http://www.jos.org.cn/1000-9825/7726.htm>

Threat-intelligence-driven Multi-agent Android Anti-runtime Analysis Method

YAN Xin-Peng¹, ZHANG Xiang-Yu^{1,4}, FAN Ling-Ling^{1,4}, SU Yu-Cheng³, CHEN Sen¹, LIU Zhe-Li¹, CAO Xiao-Chun^{2,4}

¹(College of Cryptology and Cyber Science, Nankai University, Tianjin 300350, China)

²(School of Cyber Science and Technology, Sun Yat-sen University, Shenzhen 518107, China)

³(Xiaohongshu, Shanghai 200031, China)

⁴(Zhongguancun Academy, Beijing 100094, China)

Abstract: The openness and fragmentation of the Android ecosystem expose it to continuously evolving and increasingly complex security threats. An increasing number of malicious attackers use runtime analysis techniques to probe application logic and tamper with execution

* 基金项目: 京津冀自然科学基金合作专项 (25JJJC0003); 北京中关村学院项目 (20240302)

严欣芃和张向宇为共同第一作者.

收稿时间: 2025-12-28; 修改时间: 2026-03-10, 2026-05-27; 采用时间: 2026-06-23

flows, thus bypassing Android security measures and increasing the persistence and impact of malicious activities. Anti-runtime analysis technology detects runtime analysis activities, enabling users and developers to identify such activities promptly and take protective measures, thus safeguarding application integrity and data security. Currently, various anti-runtime analysis methods based on static databases have been proposed. These methods collect threat intelligence on the characteristics of known runtime analysis techniques and match information extracted from devices against this intelligence to detect traces of runtime analysis techniques. However, these methods suffer from “knowledge lag” and provide insufficient analytical depth and result interpretability, making them increasingly ineffective against rapidly evolving runtime analysis techniques. To address these issues, this study proposes and implements AIntel-Agent, a threat intelligence-driven multi-agent Android anti-runtime analysis method. The proposed method uses agent technology to build a dynamic knowledge base. Device fingerprints are then analyzed in depth based on the characteristics of five categories of runtime analysis techniques, including rooting, hooking, debugging, virtual environments, and tampering, thus enabling Android anti-runtime analysis tasks to be completed in an automated and intelligent manner. Through the design of an agent collaboration framework, AIntel-Agent proposes a method for constructing a dynamic knowledge base by obtaining threat intelligence from the Internet in real time and develops an iterative reasoning mechanism that associates device fingerprints with the characteristics of runtime analysis techniques to generate anti-runtime analysis reports for devices. Experimental results based on a dataset of real devices show that AIntel-Agent achieves a precision of 88.5% and a recall of 93.9% on Android devices. Its risk reports contain sufficient evidence and are interpretable, demonstrating its effectiveness in Android anti-runtime analysis.

Key words: Android anti-runtime analysis; large language model (LLM); multi-agent system; software analysis; mobile security

Android 系统凭借其开放性和灵活性, 已在全球智能手机市场占据主导地位, 根据 StatCounter 提供的统计数据, 截至 2025 年 10 月, Android 操作系统在全球移动操作系统中占据 72.55% 的市场份额^[1], 这种开放的生态系统也使 Android 系统成为攻击的首要目标. 近年来, 针对 Android 平台的攻击持续呈现高发态势, 其手段日益复杂和隐蔽^[2,3]. 恶意攻击者开始使用运行时分析技术, 如动态 Hook、指令插桩及协议分析等方式绕过安全防护, 通过隐蔽的运行环境 (如定制化只读内存 (read only memory, ROM)、隐藏 Root、虚拟化容器) 来规避检测, 从而窃取应用的核心算法与知识产权, 实现恶意爬虫、伪造业务请求等非法获利行为. 为了维护商业利益与数据安全, 应用开发者广泛采用反运行时分析 (anti-runtime analysis) 技术作为保护知识产权及遏制恶意篡改的重要手段^[4]. 通过部署反运行时分析技术, 开发者可以发现设备上存在的运行时分析技术的痕迹, 及时采取防护措施, 避免核心逻辑泄露与恶意篡改的风险. 然而, 运行时分析技术在实现方式与隐蔽性方面的发展使现有的反运行时分析技术愈发难以识别. 因此, 如何获取最新运行时分析技术的情报, 实现可靠的反运行时分析, 保障 Android 生态安全, 已成为亟待解决的问题.

Android 反运行时分析旨在通过分析从 Android 设备中提取的指纹特征与运行时分析技术的威胁情报, 识别当前设备上存在的 Root、Hook、调试 (debugging)、虚拟环境 (virtual environment)、篡改 (tampering) 等运行时分析技术^[5-7], 使用户与开发者能及时采取反制措施, 规避后续风险. 目前 Android 反运行时分析的主要方式基于静态知识库分析^[7-12], 这类方法通过收集已知运行时分析技术的威胁情报, 即技术的实现原理、环境需求、运行特征等信息, 构建一个静态知识库, 使用关键词匹配的方式将 Android 设备指纹信息与知识库中存储的特征进行匹配, 从而识别设备上存在的运行时分析工具.

然而, 目前的 Android 反运行时分析方法存在两个缺陷: 1) 静态知识库固有的“知识滞后”问题. 面对知识库更新截止后的特征, 传统方法的有效性会显著下降^[2,3]. 而面对高度迭代的各类运行时分析工具, 维持一个内容全面、实时更新的知识库近乎不可能. 2) 分析深度与可解释性不足. 目前已有的反运行时分析工作主要使用关键词匹配的方法, 将特定的指纹特征与静态知识库中已知的运行时分析工具特征进行匹配, 攻击者通过对少量特征的伪装即可规避. 同时, 基于关键词的匹配难以捕捉指纹特征的深层语义信息, 也无法给出完整的推理证据链^[13,14]. 这些缺陷使传统方法难以实现持久有效的反运行时分析工作.

近年来, AI 智能体 (AI agent) 技术的出现显著提升了将大语言模型 (large language model, LLM) 应用在专业领域任务时的效果^[13-17]. 智能体能够与开放网络交互, 且具备真正的语义理解能力. 通过网络搜索, 智能体可以获取网络中的最新情报, 从而构建一个实时更新的动态知识库, 语义理解能力使智能体能够认知各项指纹属性的意

义,用证据支撑自身做出的判断,这为解决传统方法的缺陷带来新的可能。但是,目前基于智能体技术的 Android 反运行时分析仍面临两项挑战:1)全面获取知识并进行验证。反运行时分析任务需要全面的运行时分析技术知识作为推理基础,智能体的网络搜索能力不足以给出覆盖全部运行时分析技术的结果,更难以对结果进行验证。如何让智能体获取足够全面的信息,并通过验证与处理将其转化为结构化的知识,是一项关键挑战。2)引导智能体理解知识,进行推理。在执行反运行时分析任务时,智能体难以理解指纹痕迹的成因,无法有效从分散的指纹痕迹中推理出运行时分析技术存在的证据。如何引导智能体真正理解运行时分析技术与指纹痕迹的关系,进行逻辑清晰的推理,最终给出令人信服的证据是一个难题^[17]。

因此,本文提出基于威胁情报驱动的多智能体 Android 反运行时分析方法 AIntel-Agent (Android intelligent risk evaluation multi-agent system)。AIntel-Agent 使用多智能体系统,实时获取网络中运行时分析相关的威胁情报,从 Root、Hook、调试、虚拟环境、篡改这 5 类运行时分析技术的特征出发对 Android 设备指纹进行分析,发现设备中可能存在的运行时分析技术。AIntel-Agent 设计了 4 个不同的智能体以系统性克服上述挑战:针对知识获取与利用的挑战,威胁研究规划员与风险知识检索员协作,将 Android 设备面对的运行时分析威胁拆解并形成一份搜索计划,根据计划在网络上全面地收集威胁情报并总结为结构化的风险标签,从而构建一个动态威胁情报知识库。通过网络实时获取最新威胁情报,AIntel-Agent 有效地解决了静态知识库知识滞后的问题。对于智能体理解专业知识的挑战,指纹风险分析员通过循环推理机制,深入分析设备指纹,通过网络搜索补充知识,从而形成对设备指纹的认知基础,将指纹中离散的异常痕迹与动态知识库中的风险标签关联,给出一份逻辑严密的反运行时分析报告。通过智能体的循环推理,AIntel-Agent 能够真正理解设备指纹的含义,整合离散的风险特征,在报告中给出逻辑清晰的证据链,从而保证分析的深度与结果的可解释性。智能体协调员负责整体流程的调度与结果评估,它为智能体分配任务,审查智能体返回的结果是否满足需求,动态调整工作方向。通过不同智能体的协作,AIntel-Agent 可以利用网络实时获取最新威胁情报,对设备指纹进行分析与推理,将隐蔽、分散的痕迹结合起来,识别出运行时分析技术的存在,高效准确地完成 Android 反运行时分析工作。

基于实验评估,AIntel-Agent 在人工构建的数据集和现实 Android 设备数据集上分别达到 97.7% 和 88.5% 的精确率,100% 和 93.9% 的召回率,展示出该方法在无静态知识库背景下仍具备准确反运行时分析的能力。通过对比不同 LLM 对 AIntel-Agent 性能的影响,我们确认了 GPT-4o 与 Claude-sonnet-4 混合使用能使 AIntel-Agent 的反运行时分析达到最佳效果。实验结果显示,多智能体系统相比单智能体在精确率上提升了 17.2%,召回率上提升了 18.3%,证明多智能体系统在执行 Android 反运行时分析任务中的有效性,系统中各智能体均对 AIntel-Agent 的整体性能产生积极作用。

总体而言,本文的主要贡献如下。

- 提出基于威胁情报驱动的多智能体 Android 反运行时分析方法 AIntel-Agent,实时获取网络中的威胁情报,从 Root、Hook、调试、虚拟环境、篡改这 5 个方面分析设备指纹中的运行时分析技术痕迹,有效解决传统 Android 反运行时分析中知识滞后、分析深度与可解释性不足的缺陷。
- 使用多智能体系统,通过智能体协调者分解任务、动态协调工作,威胁研究规划员生成计划指导工作流程展开,风险知识检索员从网络搜集威胁情报构建动态知识库,指纹风险分析员深度分析设备指纹、关联风险并产出报告,有效地完成 Android 反运行时分析任务。
- 基于人工数据集和现实设备数据集设计实验,证明 AIntel-Agent 在 Android 反运行时分析任务上的有效性,对比使用不同 LLM 执行任务的性能,多智能体系统相比单智能体的综合性能提升 24.1%。
- 提出通过多智能体系统自主进行网络搜索构建动态知识库,深度分析设备指纹进行反运行时分析的方法,能够为未来 Android 设备反运行时分析及相关领域的工作提供支持。相关资源开源在 GitHub 中,可在 <https://github.com/liyao-l-y/AIntel-Agent> 获取。

本文第 1 节介绍 Android 设备反运行时分析的相关研究。第 2 节介绍 Android 设备指纹和 Agent 相关基础知识。第 3 节介绍 AIntel-Agent 的总体设计与实现细节。第 4 节介绍实验设计、评估指标与结果分析。第 5 节讨论

AIntel-Agent 的局限性和发展方向. 第 6 节对本文工作进行总结.

1 相关工作

随着移动互联网的普及, Android 操作系统已占据全球智能手机市场的主导地位. 然而, Android 开放的生态系统也使其成为网络攻击的首要目标. 近年来, 针对 Android 设备的攻击变得日益复杂和频繁, 仅在 2022 年, 就有超过 348 万个恶意应用被识别和移除, 针对 Android 应用的攻击在 2020–2023 年间增长了 312%^[18]. 开发者使用的反运行时分析与攻击者使用的运行时分析工具在对抗中不断迭代, 这种动态演变的威胁形势对传统的反运行时分析方法提出了严峻挑战. 接下来, 我们回顾代表性的 Android 反运行时分析方法, 并关注基于 LLM 与 Agent 技术的相关工作, 以启发下文的方法设计思路.

1.1 Android 反运行时分析

无论是开发者为了保护应用的代码与知识产权, 还是恶意软件为了隐藏自身行为并规避安全审计, 应用都希望感知自身是否正受到运行时分析, 并通过改变执行行为来避免程序逻辑暴露. 围绕这一问题, 已有研究从多个角度进行了探索. 已有工作表明, 运行时分析工具本身会在执行过程中留下可被应用感知的痕迹.

例如, Apostolopoulos 等人^[7]在 Windows 环境下利用内存重映射技术, 将磁盘中的原始程序映射到内存作为对照副本, 并通过比较当前内存内容与原始副本之间的差异检测 Hook 痕迹. 这类研究说明, 分析工具引入的代码修改、内存篡改与执行语义变化, 本身就可能成为应用识别分析者的重要依据. 但此类工作主要面向桌面平台, 而非 Android 场景下反运行时分析能力的实现与识别问题. 在 Android 场景下, 相关研究更多聚焦于特定类型的环境感知. Hong 等人^[12]通过比较应用在不同运行环境中的执行轨迹差异, 定位导致行为分歧的程序点及其逻辑约束, 从而识别应用中的环境判别逻辑. Kondracki 等人^[11]则提取通话记录、媒体文件数量、系统设置等环境特征, 并利用随机森林模型区分真实设备与虚拟环境. 前者关注应用内部是否存在与环境相关的条件分支, 后者关注设备环境是否接近真实用户场景, 二者都为理解 Android 应用的环境感知提供了重要参考, 但并未直接回答设备当前是否存在 Hook、调试、篡改或其他运行时分析风险. 而最新的相关工作通过检测特定文件路径、应用包名、系统属性或 Root 痕迹, 判断设备是否被 Root 或是否安装了已知分析工具^[10]. 这类方法实现较为直接, 但通常建立在预定义规则或显式指纹之上, 其本质仍是对已知特征的匹配, 因而存在明显的知识滞后问题.

而 Android 恶意软件检测领域的相关研究^[2,3]进一步表明, 在复杂 Android 环境下, 传统上建立在显式特征和分类边界之上的 ML 模型, 同样可能被攻击者通过对 APK 属性进行简单修改而干扰输出. 因此, 业界亟需一种能够真正理解 Android 环境下运行时分析痕迹语义、可动态更新知识库的检测方法.

1.2 LLM 和 agent 在安全任务中的应用

大语言模型 (LLM) 的出现, 为解决基于静态知识库的反运行时分析方法中的问题带来了新的可能. 凭借 LLM 强大的逻辑推理能力和工具调用能力, 以 LLM 为核心的智能体 (agent) 能够通过网络搜索^[19,20]动态获取外部知识, 并理解设备指纹与潜在运行时分析技术之间的复杂联系. 这使得自动化、智能化的安全分析成为可能, 并已在一些研究中得到初步探索^[13]. 然而, 现有研究同样揭示了将单一 LLM agent 直接应用于复杂专业领域时所面临的瓶颈^[21]. 首先, 简单地引入网络搜索并不能保证信息的可靠性, LLM 如何有效利用检索内容也是一个难题^[19,20]. 其次, 在执行如 Android 反运行时分析这类涉及多步骤、长链条推理的复杂任务时, 单一智能体存在能力上的局限性, Xing 等人^[21]的研究系统性地指出单一 LLM agent 在理解、推理、探索和反思这 4 种核心能力上的不足. 为解决这些缺陷, 研究者们使用多智能体系统为 agent 设计专门的协作框架, Shao 等人^[14]利用了多视角提问的协作模式, 引导多个 agent 从不同角度对主题进行深入探究和信息搜集. Tang 等人^[15]通过多个专用 agent 协调执行零代码的自动化开发工作. Hu 等人^[16]通过 Planner、Coordinator 和 Worker 这 3 个层级, 构建由多个 agent 组成的协作团队来完成任

务. 在 Android 安全方面, AgentDroid^[17]通过 8 个负责特定任务的智能体模拟专家团队, 通过 1 个智能体动态分配任务, 6 个专业智能体处理应用的多模态数据, 进行应用属性的关联与汇总, 由 1 个智能体进行决策, 实现对

Android 应用欺诈行为的检测. LiveSCA^[22]通过智能体协作, 从应用中提取特征, 并在网络上收集第三方库的信息, 随后验证搜索结果的可靠性, 将验证后的信息与应用特征聚合, 实现对 Android 应用的成分分析. 受以上工作的启发, 本文提出基于多智能体协作的 Android 反运行时分析方法 AIntel-Agent, 将复杂的 Android 反运行时分析任务分解, 并分配给角色明确、能力互补的多个智能体协同完成, 通过智能体协调员动态分配任务, 由威胁研究规划员与风险知识检索员从 Root、Hook、调试、虚拟环境、篡改这 5 个方面收集 Android 运行时分析技术的威胁情报, 指纹风险分析员负责分析设备指纹, 将异常痕迹与威胁情报相关联, 最终完成整个反运行时分析任务.

2 基础知识

2.1 Android 设备指纹

Android 设备指纹是指一系列能够描述设备软硬件特征的集合. 它并非单一的数据, 而是通过采集设备在运行时暴露出的多维度属性, 综合构成数据集合.

设备指纹采集的环境可以总结为以下 3 种层面^[23-25]: (1) 硬件与系统层信息: 如 CPU 型号、内存大小、屏幕分辨率、各类传感器信息等硬件参数, 以及系统版本、内核版本、品牌型号、编译信息等固化在系统中的属性; (2) 软件与应用层信息: 如设备应用列表、系统证书列表、特定风险文件与权限状态等; (3) 状态与行为层信息: 如设备是否开启开发者模式、是否处于 USB 调试状态、是否处于模拟器环境等. 设备指纹是互联网风控与反欺诈领域的核心技术基石, 其根本目的在于确认设备的运行环境, 从而为后续的决策提供依据.

2.2 Android 运行时分析技术

Android 运行时分析 (runtime analysis) 技术是指主动追踪目标应用或改变设备运行环境以进行动态分析、代码调试或逆向工程的一类技术. 该类技术在被恶意攻击者利用时会成为危害设备安全与规避检测的重要手段. 攻击者通过嵌入环境判断与自保护逻辑, 使得仿真器、Hook 技术或 Root 环境应体现的特征被抑制或改变, 从而增加检测成本并延长恶意代码的存活期. 理解这些运行时分析技术及其实现细节, 对于构建可靠的反运行时分析策略具有重要意义.

Android 平台上的运行时分析技术可以分为以下 5 种类型^[4]: 一是 Root, 攻击者利用系统漏洞或解锁 BootLoader 来获取 Root 权限, 为深度修改系统铺平道路; 二是 Hook, 攻击者借助 Xposed、Frida 等 Hook 框架, 拦截并修改应用及系统中关键方法的返回值, 向应用层提供虚假的设备指纹信息实现欺诈; 三是调试 (debugging), 攻击者通过调试器和动态逆向工具对应用或系统的关键运行过程进行分析, 随后通过插桩等手段展开攻击; 四是虚拟环境 (virtual environment), 在 PC 或服务器上运行的 Android 模拟器以及云手机, 它们在软件层面模拟完整的 Android 环境, 能以极低成本生成海量看似全新的设备, 常被用于批量注册虚假账号和执行自动化脚本; 五是篡改 (tampering), 攻击者篡改应用的关键代码/资源, 通过重签名或从非信任来源安装的方式将其伪装成正常应用. 这些运行时分析技术的实现形式多样、更新迅速, 且常以组合形式出现, 给静态与动态分析都带来挑战. 本文提出的 AIntel-Agent 从这 5 个方面对设备指纹运行时分析技术的痕迹进行分析.

反运行时分析 (anti-runtime analysis) 技术, 即通过检测设备指纹中存在的 Root、Hook、调试、虚拟环境、篡改这 5 类运行时分析技术遗留的痕迹, 发现设备中存在的运行时分析技术, 从而使用户与开发者能及时发现风险并采取防护措施, 阻止后续的恶意行为.

2.3 多智能体系统

大语言模型智能体 (LLM agent) 是以 LLM 为核心, 通过规划 (planning)、记忆 (memory) 和工具使用 (tool use) 等核心部件的协同作用, 完成复杂任务的自主系统^[13]. LLM agent 能够将宏大目标分解为可执行子任务, 并调用搜索引擎、API 等外部工具来获取实时信息或执行特定操作, 从而突破 LLM 自身知识的局限性.

多智能体系统 (multi-agent system) 为克服在复杂、动态和专业化场景中单一智能体的能力局限^[21]而诞生. 多智能体系统的核心思想是将一个复杂的任务分解为一系列更易于执行的子任务, 分配给多个角色明确、能力互补

的 agent 协同完成. 通过构建结构化的协作框架, 可以让不同 agent 进行交互, 规划各自的任务并对结果进行评估, 从而改善信息获取的效果, 自适应地解决任务执行中的冲突与难题^[20]. 在组织 agent 的协作流程时, 通常采用动态工作流的方式. 动态工作流允许 agent 根据任务执行过程中的实时反馈和新增知识, 动态地规划、调整和重新配置任务结构. 这种动态性使系统能够灵活应对开放环境中复杂的研究型任务^[14,16,26,27]. 通过多智能体协作来构建动态工作流, 不仅能深化 LLM 的特定能力, 还能通过结构化协作来提升整个流程的稳定性和鲁棒性. 受此启发, 本文设计了 AIntel-Agent, 将复杂的 Android 反运行时分析任务分解到多个专门的智能体, 通过它们之间的协作来克服单一智能体在执行动态工作流时的能力瓶颈, 提升整体分析的深度和准确性.

2.4 研究动机

我们调研了目前进行 Android 运行时分析时主要使用的 Root 和 Hook 框架 (例如 LSPosed^[28]、APatch^[29]和 KernelSU^[30]) 在 GitHub 上的更新频率. LSPosed 项目的首个版本发布于 2021 年 2 月 2 日, 截至停止维护, LSPosed 共发布了 39 个版本, 整体平均更新周期 25 天, 特别是项目刚发布的半年内平均 10 天就有一个新版本. APatch 的首个公开版本于 2023 年 12 月 31 日发布, 至今发布了 43 个版本. APatch 在项目刚推出的 2024 年第 1 季度, 几乎每天都有新版本, 呈现“高速迭代”状态. 整体来看 APatch 的平均更新周期为 12 天. KernelSU 的首个正式版本发布于 2023 年 1 月 28 日, 从 2023 年初至今共发布 38 个版本. KernelSU 发布高峰集中在 2023 年上半年, 当时平均每月发布约 3 个版本. 综合整个生命周期, KernelSU 平均更新周期为 20 天. 这些数据清晰地表明, 攻击工具的开发者正以极高的频率改进其技术. 每一次更新都可能意味着旧有检测逻辑的失效. 一个依赖固定知识库的防御系统, 在面对平均更新周期仅为两周的对手时, 其防御策略将永远处于被动追赶和过期失效的循环中.

其次, 攻击工具的迭代只是冰山一角, 更严峻的挑战在于围绕这些工具形成的知识社区, 这会导致威胁知识与攻击手法的传播. 我们调查了国内社区 (如酷安^[31])、国外社区 (如 Reddit^[32]) 上多个与 Android 安全相关的版块活跃度. 这些社区平均每隔几小时就会出现一篇新帖, 社区用户针对厂商推出的检测与防御方式, 不断开发攻击手段与隐藏技巧. 对于防御而言, 这带来了巨大的情报处理压力: 如何在海量的互联网知识中, 快速、准确地捕捉到真正构成威胁的关键情报, 并将其转化为可执行的防御策略, 是当前依赖人工分析的模式难以解决的困境.

此外, 当前针对运行时分析的检测能力也面临严峻挑战. 我们使用目前存在的闭源设备环境检测工具对设备进行实际检测, 发现现有的 Root 工具仅通过基础的伪造自身包名、随机化进程名等方式, 即可使现有检测工具给出的风险等级出现明显下降. 以闭源环境检测工具 Momo^[33]为例, 我们在使用 Magisk 进行 Root 的设备上进行测试, 在不开启任何隐藏方式的情况下, 其可以报告出 BootLoader 解锁、检测到 su 命令、检测到 Magisk 包名等信息, 判断当前设备存在高风险. 当开启随机包名等基础的隐藏方式后, Momo 仅能检测出 BootLoader 解锁这一信息, 设备风险也降低至存在可疑痕迹, 无法检测到 Root 工具的存在.

如果攻击者使用专用的隐藏组件, 能够进一步深度掩盖运行时修改痕迹, 使这些传统的闭源检测工具几乎失效, 最终仅能发现如开发者选项、USB 调试等边缘风险. 这同样表明, 现有方法对于深度隐藏的对抗性痕迹缺乏有效的深层挖掘与关联推理能力.

3 AIntel-Agent 方法设计

为克服单一智能体在长链条、知识密集型任务中的局限性, 本文设计并实现了一个基于威胁情报驱动的多智能体 Android 设备反运行时分析方法——AIntel-Agent. 该方法的核心思想是将复杂的反运行时分析任务动态分解为一系列由角色明确、能力互补的专业智能体执行的子任务. 整个协作流程由一个作为“大脑”的智能体协调员进行统一规划、调度与整合, 并通过一个“思考→行动→反思”的循环推理机制来驱动, 模拟人类专家团队解决问题的动态协作过程. 如图 1 所示, AIntel-Agent 以智能体协调员为中心, 接收一份设备指纹采集结果作为输入. 协调员负责调度 3 类核心的专业智能体协同工作, 最终输出一份高质量、可解释的反运行时分析报告.

- 智能体协调员: 负责任务的整体规划、分解、调度以及对最终结果的整合与决策.
- 威胁研究规划员: 负责将运行时分析技术类别转化为具体的搜索方向, 制订一份威胁情报搜索计划.

- 风险知识检索员: 负责根据搜索计划执行网络搜索、结果验证和结构化信息提取, 构建动态知识库。
- 指纹风险分析员: 负责对设备指纹与威胁情报进行循环推理并关联, 识别潜在的运行时分析技术。

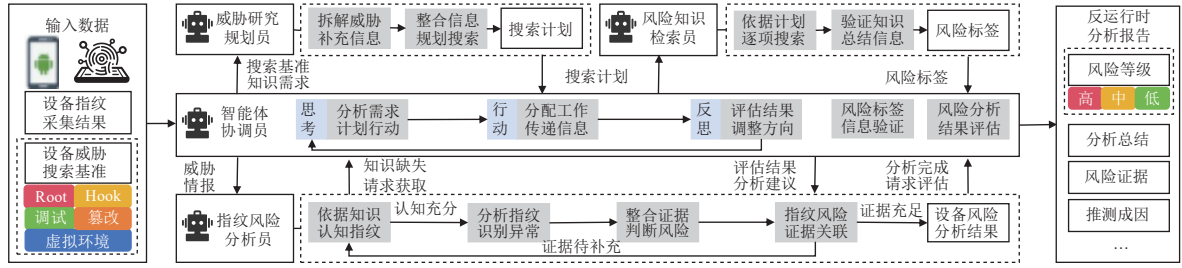


图 1 AIntel-Agent 方法设计

为了克服全面获取知识并进行验证的挑战, 我们设计了威胁研究规划员与风险知识检索员, 前者根据智能体协调员给出的信息获取需求构建一份搜索计划, 保证搜索的全面性; 后者根据搜索计划从网络实时获取最新的威胁情报, 总结出风险标签, 由智能体协调员进行验证后构成动态知识库。对于智能体理解专业知识并与设备指纹关联的难点, 本文设计了指纹风险分析员, 它先结合动态知识库认知设备指纹属性的来源与意义, 主动补充知识缺陷, 然后通过循环推理识别设备指纹中存在风险或互相冲突的属性, 将离散的异常特征与风险标签相关联, 构建起证据链, 由智能体协调员对分析结果进行评估后, 给出设备反运行时分析报告。与线性的处理流程不同, AIntel-Agent 的协作模式是动态和自适应的。协调员能够根据任务执行过程中的实时反馈灵活地调整策略、重新调用智能体, 从而显著提升整个分析流程的鲁棒性和准确性。

3.1 智能体协调员

智能体协调员是 AIntel-Agent 的核心, 它不直接执行具体任务, 而是通过一个“思考→行动→反思”的循环推理机制来串联起 AIntel-Agent 的整个工作流程。这种机制赋予了工作流程中规划、执行和自我修正的能力。在任务过程中, 智能体协调员会评估当前的任务完成情况、专业智能体给出的结果以及当前需要完成的目标, 制定下一步的行动策略。智能体协调员将反运行时分析任务分解为搜索规划、网络搜索、指纹分析等清晰且易执行的子任务, 为工作流程提供明确的方向; 通过“反思”环节, 智能体协调员赋予了 AIntel-Agent 应对未知和不确定性的能力, 当专业智能体遇到知识缺陷时, 协调员能够动态地调整策略, 例如重新启动一轮威胁情报搜集, 确保了整个多智能体系统能够作为一个有凝聚力的整体, 高效、有序地向最终目标推进。

在任务的初始阶段, 智能体协调员需要将 Android 运行时分析技术转化为具体的技术威胁。它的思考过程是: “当前目标是分析设备指纹, 但缺乏最新的威胁情报。因此, 我需要提出一份研究计划。我将从已知的运行时分析技术类别中推理出具体的技术威胁, 并将其交给威胁研究规划员”。这种推理能力使得协调员能够将模糊的高层目标转化为清晰的子任务。根据思考阶段制定的计划, 协调员会调用专业智能体执行具体的子任务。协调员接收并评估专业智能体返回的结果。如果结果满足预期, 则进入下一阶段或终止任务。如果结果不满足预期 (如风险标签信息来源不可信、分析报告证据不充分), 协调员会总结问题, 并带着新的认知重新进入“思考”阶段, 以启动新一轮的修正循环。为防止无限循环, 本文为思考步骤的最大次数设置了阈值 (默认为 5)。通过这种结构化的循环, 协调员能够高效模拟人类专家团队解决复杂问题的迭代式工作流。后文图 2 展示了智能体协调员的关键提示词。其中, 通过提供上下文 (context) 和少样本提示 (few-shot example) 来引导协调员的行为, 确保其能够正确地启动和推进任务。

3.2 基于 LLM 的专业智能体

为了确保每个专业智能体都能高效、可靠地完成其任务, 我们采用人格模式 (persona pattern) 和分步思考 (step-by-step thinking) 的提示工程策略^[13-15]。每个智能体的行为都由一个精确的、结构化的提示词驱动, 提示词定

Input:
Role & Goal
You are an expert-level Agent Coordinator for Android security analysis. Your goal is to orchestrate a team of specialized agents to analyze a given device fingerprint report and produce a comprehensive risk assessment.

Context: System Background Knowledge
You have access to the following established knowledge about Android device risks:...

Core Mechanism: Thought-Action-Reflection Cycle
To solve the task, you must follow a structured approach with interleaving Thought, Action, and Reflection steps.

1. Thought: Reason about the current state and determine the next action. Your reasoning should be concise and goal-oriented.
2. Action: Invoke one of the specialized agents to perform a task. Use the following syntax:...
3. Reflection: Evaluate the output from the agent. Decide if the goal is met or if another cycle is needed. Summarize your findings and learnings.

Few-shot Example (Updated to reflect the new logic)
Task: Analyze fingerprints for a device.
LOOP:
Thought: 1. Initial Goal 2. Prerequisite Check 3. Sub-Goal 4. Reasoning on Input Categories 5. Action Plan
Action: Planner(categories=["Emulator Indicators", "Cloud Phone Fingerprinting", "Privilege Escalation (Rooting)", "Dynamic Injection (Hooking)", "Automation Frameworks"])
Reflection: The Planner returned a detailed search plan. Next, I will use this plan to retrieve knowledge.
...
Final Answer: [The final risk report generated by the Analyst]

Current Task
Input: {device_fingerprint_json}
Your turn. Start with "Thought".

角色与目标
你是一名专家级的Android安全分析协调员。你的目标是协调一支专业团队，分析给定的设备指纹报告，并提供全面的风险评估。

背景知识: 系统背景知识
你拥有关于Android设备风险的以下已建立知识： ...

核心机制: 思考-行动-反思循环
为了解决任务，你必须遵循结构化的方法，交替进行思考、行动和反思步骤。

1. 思考：推理当前状态并确定下一步行动。你的推理应简明扼要且以目标为导向。
2. 行动：调用专业代理执行任务。使用以下语法： ...
3. 反思：评估代理的输出。决定目标是否达成，或者是否需要另一个循环。总结你的发现和学习。

少量示例
任务：分析设备的指纹。
循环：
思考：1. 初始目标 2. 前提检查 3. 子目标 4. 输入类别推理 5. 行动计划
行动：策划者 (类别=["仿真器指标", "云手机指纹识别", "特权升级 (越狱)", "动态注入 (钩子)", "自动化框架"])
反思：策划者返回了一个详细的搜索计划。接下来，我将利用该计划获取知识。
...
最终答案: [分析师生成的最终风险报告]

当前任务
输入：{设备指纹_json}
进行你的行动，以“思考”开始。

3.2.1 威胁研究规划员

AI-Agent 需要的威胁情报为 Root、Hook、调试、虚拟环境、篡改这 5 个类别的运行时分析技术信息。智能体协调员将其传递给威胁研究规划员。如表 1 所示, 威胁研究规划员会从这些类别出发, 推理出 5 个最有利于搜索展开的关键词。例如对于“Hook”, 智能体会推理出“Frida Android 14 Hook bypass”等 5 个关键词。这些关键词可以引导后续智能体对多种不同技术进行搜索, 提升知识获取的效率与覆盖度, 确保智能体能够锁定最新的、关键的威胁信息。其关键提示词如图 3 所示。

表 1 搜索关键词示例

威胁	关键词
Root	Android 15 Root exploit, KernelSU Android 14 bypass, Magisk alternative Root methods, Android BootLoader unlock vulnerability, CVE Android privilege escalation Root
Hook	Frida Android 14 Hook bypass, LSPosed Xposed framework alternative, Android ART runtime Hooking, IL2CPP Unity game Hooking Android, Android native library injection techniques
debugging	Play Integrity API bypass, SafetyNet attestation spoofing methods, Android Root detection bypass tools, banking app security bypass techniques, Google Play Protect disable methods
virtual environment	Android accessibility service automation, Appium mobile automation framework, Android UI testing automation tools, mobile game bot development frameworks, Android device farm automation solutions
tampering	Android build.prop editor Root, device fingerprint spoofing tools, Android system property modification, mobile device emulation detection bypass, Android hardware ID changer tools

Input:
Role & Goal
You are a "Threat Research Planner" specializing in Android security. Your goal is to create a comprehensive and actionable search plan to gather intelligence on modern Android attack vectors like Rooting, Hooking.
Input
- categories: A list of high-level technical threat categories provided by the Coordinator.
Instructions
Let's think step by step.
1. Brainstorm Keywords: For each threat category, brainstorm a diverse set of specific and effective search keywords. Include names of popular tools (e.g., Magisk, Frida), techniques (e.g., "SafetyNet bypass"), and relevant vulnerabilities (e.g., "CVE-2023-xxxx Android").
2. Structure the Plan: Organize these keywords into a structured plan. The plan should guide the next agent (Retriever) to search systematically, ensuring broad coverage and avoiding redundancy.
3. Prioritize: If possible, suggest a priority for the search topics based on their prevalence and impact.
Output Format
Provide the output as a JSON object, detailed in the appendix.

角色与目标
你是一名专注于Android安全的“威胁研究规划师”。你的目标是创建一个全面且可执行的搜索计划，以收集关于现代Android攻击向量的情报。
输入
categories: 由协调员提供的高层次技术威胁类别列表。
指令
让我们一步步思考。
1. 头脑风暴关键词：针对每个威胁类别，头脑风暴一组多样化的具体和有效的搜索关键词。包括流行工具的名称、技术和相关漏洞。
2. 组织计划：将这些关键词组织成一个结构化的计划。该计划应指导下一个智能体进行系统性的搜索，确保广泛覆盖并避免冗余。
3. 优先级：如果可能，建议根据主题的普遍性和影响程度为搜索主题设定优先级。
输出格式
提供一个JSON格式的输出，示例如下：
{ "搜索计划": [{
 "类别": "权限提升",
 "威胁等级": "高",
 "关键词": ["Magisk alternative 2024", "KernelSU Android 14", "Android bootloader unlock vulnerability"] }, ...] }

图 3 威胁研究规划员的关键提示词

3.2.2 风险知识检索员

在开放网络的搜索结果中会存在过时内容甚至虚假信息, 直接将原始数据作为分析依据会影响最终结论的准确性. 为了缓解这一问题, 本文设计了风险知识检索员, 将网络搜索的结果总结为结构化的风险标签并由智能体协调员进行验证, 确保所构建的动态知识库的可靠. 风险知识检索员扮演着“研究助理”的角色, 获取原始的网络数据后, 进行清洗和转化, 为后续智能体提供可以直接使用的高质量情报. 它接收规划员制定的搜索计划, 调用外部工具进行网络搜索, 其核心任务是将非结构化的网络文本信息, 提炼和转化为结构化的“风险标签”, 形成动态知识库. 动态知识库中的风险标签中包含风险描述、技术细节和信息来源, 在提炼风险标签时, 风险知识检索员会赋予阿里云漏洞库^[34]、Android Security Bulletins^[35]、OWASP Mobile Application Security^[36]等权威来源较高权重, 基于多个信息来源中描述相符的情报进行总结, 并由智能体协调员对其进行验证, 确保风险标签真实可信. 相较非结构化的原始数据, 将高质量、结构化的知识作为输入, 可提高后续工作中智能体的任务执行效果. 其关键提示词如图 4 所示.

反运行时分析信息搜索. 在每轮循环开始时, 风险知识检索员结合搜索计划与记忆确定每次搜索的目标, 随后调用工具进行网络搜索, 获取包含网页内容及 URL 的结果. 根据搜索结果, 风险知识检索员进行思考: “我对本次

检索目标是否已经了解全面? 我还有哪些知识需要进行进一步搜索?” 根据思考结果做出对下一步搜索方向的判断. 这种搜索循环可以持续进行, 在每一轮循环的最后都会将本次的搜索目标、执行过程及结果思考更新至记忆中, 直到记忆中的知识能够覆盖搜索计划中的全部内容. 通过不同循环间的记忆关联, 风险知识检索员能够实现渐进式的认知积累, 使各检索阶段形成连贯的知识链条而非孤立的数据节点, 从而生成更优质的风险标签.

Input:

Role & Goal

You are a “Risk Knowledge Retriever”. Your goal is to execute a given search plan, validate the retrieved information, and synthesize it into structured “Risk Tags” for our knowledge base.

Input

- A search plan object from the Threat Research Planner.

Tools

- search_tool(keyword): Executes a web search.
- get_content(URL): Fetches the content from a URL.

Instructions

Let’s think step by step. For each item in the search plan:
1. Execute Search 2. Validate Sources 3. Synthesize and Structure 4. Aggregate

Output Format

Provide the output as a JSON object, detailed in the appendix.

角色与目标

你是一名“风险知识检索者”。你的目标是执行给定的搜索计划，验证检索到的信息，并将其汇总为结构化的“风险标签”以便纳入知识库。

输入

来自威胁研究规划师的搜索计划对象。

工具

search_tool(keyword): 执行网络搜索。
get_content(URL): 从URL获取内容。

指令

让我们一步步思考。针对搜索计划中的每个项目：
1. 执行搜索 2. 验证来源 3. 合成并结构化 4. 汇总

输出格式

提供一个JSON格式的输出，示例如下：
[{ “标签名称”: “b android_privilege_escalation_magisk”,
“标签描述”: “此标签用于识别 Magisk 框架的存在...”,
“技术细节”: “指标包括存在’su’和’magisk’二进制文件、引导镜像的修改以及Magisk Manager应用等.”,
“建议风险等级”: “关键”,
“风险等级原因”: “Magisk提供了Root权限, 这会破坏整个系统的安全模型.”,
“信息来源”: [“https://magisk-manager.com/”, “https://www.xda-developers.com/...”] }, ...]

图 4 风险知识检索员的关键提示词

风险标签生成. 在搜索结束后, 风险知识检索员会总结反运行时分析技术的信息, 将自然语言转换为结构化的风险标签, 构成动态知识库. 表 2 是一个风险标签的例子, LLM 从搜索结果中提炼出风险名称、描述和技术细节, 对这项风险的风险等级做出判断, 同时给出做出该判断的原因, 最后附上信息来源.

表 2 风险标签属性示例

标签属性	标签值示例
tag_name	android_privilege_escalation
description	This tag identifies threats related to privilege escalation vulnerabilities in the Android operating system. Such vulnerabilities allow attackers to gain unauthorized access to higher privilege levels, potentially compromising sensitive data or system integrity
technical_details	Privilege escalation vulnerabilities may exploit flaws in the Android framework, kernel, or third-party components. Indicators include unusual system calls, unauthorized modifications to protected files, or unexpected behavior in privileged processes
suggested_risk_level	Critical
reason_for_risk_level	Privilege escalation vulnerabilities are critical as they can be exploited to bypass security mechanisms, gain unauthorized access, and potentially control the entire device. This poses severe risks to user data and system security
information_sources	[“https://www.example1.com”, “https://www.example2.com”]

3.2.3 指纹风险分析员

指纹风险分析员是最终进行设备反运行时分析评估的智能体。简单的特征匹配难以应对隐蔽和组合式的攻击。因此, 本文设计了指纹风险分析员, 通过主动认知与循环推理的方式理解指纹, 整合离散的信息, 用充足的证据支撑自己的判断。用于支撑的证据包括运行时分析产生的特定文件, 篡改环境时需要的特定系统属性等, 可以用于证明一个运行时分析技术是否在设备上被应用过的特征。例如 Android 设备 Root 后, 应用需要 su 文件请求并获取 Root 权限, 该文件在正常设备上并不存在, 因此, 检测到 su 文件即可作为 Root 风险的证据。

指纹风险分析员模拟人类专家的认知流程, 通过认知规划、信息获取、分析处理和综合生成等阶段对设备指纹进行深度分析^[27,37-39]。首先, 指纹风险分析员基于动态知识库对设备指纹和反运行时分析技术进行认知; 对于可疑内容, 它根据认知中的疑点和缺失知识, 向智能体协调员提出信息获取请求; 形成完整认知后, 指纹风险分析员将设备指纹中分散的异常特征进行关联, 构建完整的证据链; 最后, 基于充分的证据, 将指纹特征与具体的风险标签进行可靠映射, 智能体被要求不可仅凭“缺乏正常状态”作为直接风险依据, 而是需要与其他明确的异常特征结合, 当整体的风险分数达到标准才可确认为风险存在, 并构成风险证据链。最终生成一份包含风险等级、判断依据和成因推理的反运行时分析报告。这种模拟人类专家进行归因分析的方式, 显著降低了由单一、孤立证据导致的误报, 其关键提示词如图 5 所示。

Input:
Role & Goal
 You are an expert "Fingerprint-Risk Analyst". Your goal is to meticulously analyze a device fingerprint report, correlate suspicious findings with a provided risk knowledge base, and generate a final, evidence-based risk assessment report.

Input
 - device_fingerprints: A JSON object containing the device fingerprint data.
 - knowledge_base: A list of Risk Tag objects from the Retriever.

Instructions
 Let's think step by step to conduct a deep analysis.
 1. Initial Scan & Hypothesis 2. Evidence Gathering 3. Correlate & Conclude 4. Summarize Findings

Output Format
 Provide the output as a JSON object, detailed in the appendix.

角色与目标
 你是一名专业的“指纹风险分析师”。你的目标是仔细分析设备指纹报告, 将可疑发现与提供的风险知识库相关联, 生成最终的基于证据的风险评估报告。

输入
 device_fingerprints: 包含设备指纹数据的JSON对象。
 knowledge_base: 来自检索器的风险标签对象列表。

指令
 让我们一步步进行深入分析。
 1. 初步扫描与假设 2. 证据收集 3. 对比与结论 4. 总结发现

输出格式
 提供一个JSON格式的输出, 示例如下:

```
{
  "设备总体风险等级": "存在重大安全风险",
  "总结": "设备已确认通过 Magisk 框架获取了Root权限, 这对数据完整性和隐私构成了严重威胁。",
  "风险发现": [
    {
      "风险标签名称": "b_android_privilege_escalation_magisk",
      "风险等级": "关键",
      "支持证据": [
        {
          "指纹": "file_exists/system/bin/su", "值": "true",
        },
        {
          "指纹": "ro.boot.verifiedbootstate", "值": "orange",
        },
        {
          "指纹": "selinux.status", "值": "Permissive"
        }
      ]
    },
    {
      "原因": "解锁的引导加载程序、宽松的 SELinux 状态以及常见'su'二进制路径的存在是Magisk获取Root权限框架的有力指示。"
    }
  ]
}
```

图 5 指纹风险分析员的关键提示词

在循环推理中, 指纹风险分析员将发现的证据与风险标签对应, 并进行评分, 给出一份指纹-风险映射表。评分计算如公式 (1) 所示, 若最终总得分 $R_{\text{score}} \geq T_{\text{risk}}$ (T_{risk} 为风险阈值), 则系统判定风险标签命中。

$$R_{\text{score}} = \sum_{i=1}^N V_i + M \times C_{\text{penalty}} + K \times \mu \times \mathbb{I}(S_{\text{pos}} + S_{\text{conf}} > 0) \quad (1)$$

其中, 第 1 项代表已发现的正面证据的强度, N 为检测出的正面证据总数; V_i 为第 i 项正面证据的强度, 若证据可

直接证明运行时分析技术存在 (如发现 su 文件), 则可使评分直接超过阈值, 若证据为间接风险特征, 则需要多个证据组合才可达到阈值. 第 2 项代表已识别的伪造痕迹, M 为识别到的相互矛盾的特征对数量; C_{penalty} 为伪造加权系数, 主动伪造本身就是一种强烈的恶意信号, 当发现存在伪造痕迹时也可使评分超过阈值. 第 3 项为辅助证据, K 为返回 unknown 状态的检测项数量; μ 为 unknown 状态的属性, 作为辅助证据的惩罚系数, 确保其无法主导评分; $I(X > 0)$ 为指示函数, 当且仅当系统已经收集到正面证据或矛盾证据时为 1, 无明确证据时为 0, 使 unknown 状态的特征只在已发现正面证据时才提供风险分数.

指纹风险分析员在循环推理中持续进行风险标签选取、支撑证据评分、标签命中判断、剩余特征分析的动态过程, 依次验证风险标签直至剩余的证据评分后无法达到阈值 (例如, 没有正面证据时, 只存在 unknown 属性), 循环推理流程正式终止. 如图 6 所示, 在第 1 循环中, 指纹风险分析员识别指纹中的异常特征与知识缺口, 制订相应的搜索计划; 随后, 它调用工具执行搜索, 获取新信息后再次对异常特征和知识缺口进行思考; 最后, 指纹风险分析员根据搜索结果对异常特征做出判断, 并对后续搜索提出建议. 通过第 1 循环的搜索, 记忆中增加了有关于异常特征的知识与新的搜索目标. 在第 2 循环的开始, 指纹风险分析员根据记忆调整了搜索方向, 在获取足够的信息后对支撑风险标签的证据进行评分, 所发现的 4 项正面证据与 1 项 unknown 属性评分后超过阈值, 确认了风险标签命中, 随后开始进行下一个风险的判断.

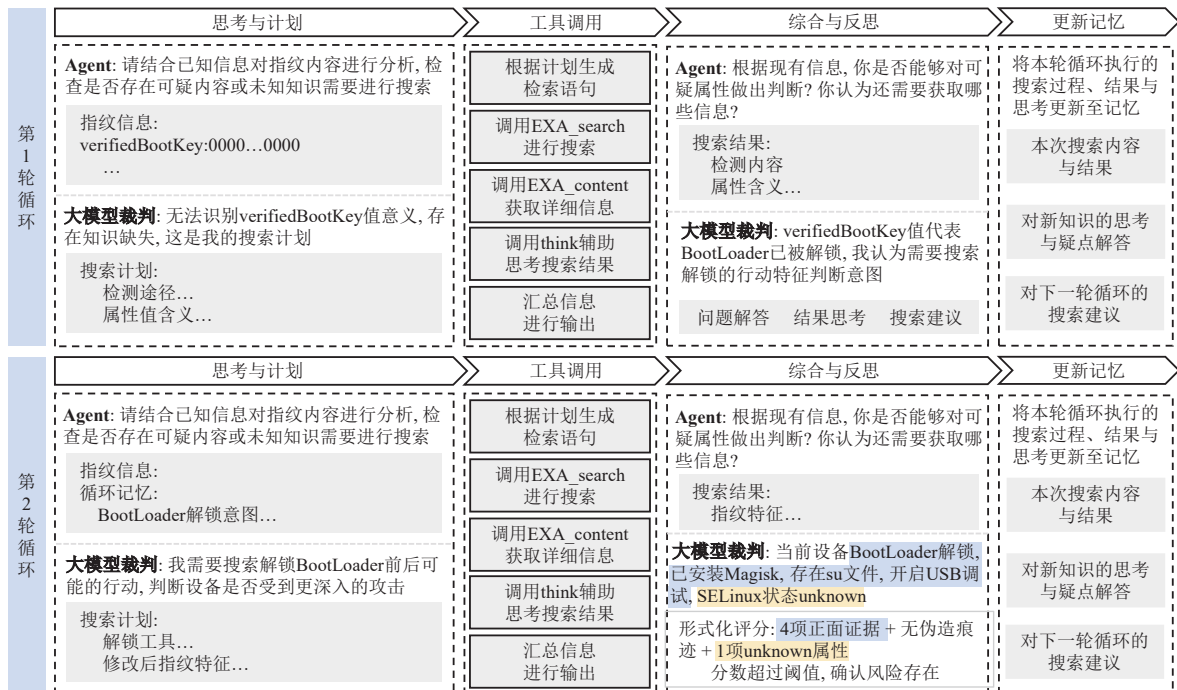


图 6 循环推理工作流程示例

随后, 根据被确认存在的风险标签的描述, 指纹风险分析员对当前存在的运行时分析工具痕迹做出评估, 形成一份报告. 报告主要包含 3 类信息: 设备的风险等级 (如 security、significant security risk present), 支持做出判断的证据, 智能体对运行时分析技术特征来源的推理. 通过这种方式, 指纹风险分析员不仅能有效识别出需要结合多个特征才能发现的复杂运行时分析特征, 更能确保最终生成的反运行时分析报告具有高度的可解释性.

4 实验分析

为评估 AIntel-Agent 进行 Android 反运行时分析任务的性能, 我们分别在自主构建的测试用例和现实设备指

纹上进行实验评估. 本文提出以下 4 个研究问题 (RQ) 以指导实验设计.

- RQ1: AIntel-Agent 对反运行时分析技术特征的识别效果如何?
- RQ2: AIntel-Agent 在现实设备指纹数据上使用不同 LLM 执行反运行时分析任务的效果有怎样的区别?
- RQ3: AIntel-Agent 的多智能体系统架构是否对分析效果有所提升?
- RQ4: AIntel-Agent 中不同智能体对反运行时分析任务的结果有怎样的影响?
- 设备指纹信息采集说明. 为了支持深度的反运行时分析, 我们参考已有检测工具的监测信息与其缺失内容, 并结合 Android 官方文档, 共设计并检测超过 170 项系统属性与环境特征. 为了防止单一维度的特征被伪造, 我们通过 Java 层与 Native 层对关键属性进行多次获取并互相交叉验证. 特别地, 本文引入由可信执行环境 (trusted execution environment, TEE) 等底层硬件生成的不可篡改信息 (如 Verified Boot 状态、Keystore 特征等), 从而为 AIntel-Agent 的后续循环分析与验证提供坚实的数据基础.
 - 环境设置. AIntel-Agent 使用 Dify 1.7.2 开发, 可以直接分析 Json 格式的指纹采集报告. 其中在未特殊说明情况下 AIntel-Agent 所使用的 LLM 为 GPT-4o+Claude-sonnet-4. 用于采集现实指纹的 Android 设备共 15 台, 覆盖主要 Android 设备品牌、型号及定制系统, 设备信息见表 3, 采集设备指纹所使用的 App 通过 Android Studio Ladybug | 2024.2.1 Canary 8 开发.

表 3 测试设备信息

设备品牌	设备型号
谷歌	Pixel 5, Pixel 6
华为	华为P50e, 华为nova10
小米	小米10S, 红米K50
vivo	vivo X100S, iQOO 10
OPPO	OPPO Reno7, 一加9R
荣耀	荣耀50, 荣耀X30

● 实验指标. 实验指标的设计旨在全面评估 AIntel-Agent 的性能, 重点关注运行时分析技术特征的识别精度. 本文使用精确率 (Precision)、召回率 (Recall) 和 $F1$ score 进行评估. 其中, 精确率用于衡量 AIntel-Agent 发现的运行时分析技术特征中真实特征所占的比例, 体现智能体通过分析避免误报的能力; 召回率反映 AIntel-Agent 在所有真实特征中成功识别的比例, 反映智能体全面识别避免漏报的能力; $F1$ score 根据精确率和召回率计算得出, 用于衡量 AIntel-Agent 在精确率和召回率同样重要的前提下的综合性能. 这些指标构成了对 AIntel-Agent 归因能力和事实一致性的全面评价标准.

4.1 AIntel-Agent 对反运行时分析技术特征的识别效果 (RQ1)

- 实验数据. 为了评估 AIntel-Agent 对于运行时分析技术指纹特征的识别效果, 我们构建了 12 个包含各种运行时分析技术和工具指纹特征的测试用例. 其中 A*代表 Android 7–Android 15 的多个 Android 版本, Root、Virtual、Hook、Mix 为测试用例中设置的运行时分析技术特征类型. Root、Hook、Virtual 表示主要包含对应的特征, Mix 表示包含 Root、Hook、调试、虚拟环境、篡改中多个类型的混合特征, 模拟现实设备测试时的不同情况. 我们为每个测试用例设置的运行时分析技术的指纹特征中包含 su 文件路径、BootLoader 解锁、签名校验结果等直接特征, 断点指令执行结果、verifiedBootKey、Java/Native 层检测一致性等间接特征, 以及高版本 Android 匿名化结果等无害特征. 本文人工标注了这些指纹特征, 以便评估 AIntel-Agent 的识别效果.
 - 实验结果. 如表 4 所示, 可以看出, Total 为 84, TP 为 84, FP 为 2, FN 为 0. 计算可得, Precision 为 97.7%, Recall 为 100%, $F1$ score 为 0.988. 对于 12 个人工构建的测试用例, AIntel-Agent 能够正确识别全部的运行时分析技术特征, 没有出现漏报. 然而, 在 A7-Root 和 A9-Hook 的两个测试用例中均出现了一次误报. 检查风险标签-指纹对应结果 (如图 7 所示) 发现, 这是由于智能体结合低版本 Android 和部分与模拟器特征相似的指纹属性, 将两个测试用例判断为处于模拟器环境中. 由于 Android 10 后隐私政策的调整, 当前常规 Android 设备与非官方模拟器

100% 的召回率识别出测试用例中存在的风险特征, 并在风险工具主动隐藏的情况下, 仍能通过指纹信息的差异识别出风险. 这证明了 AIntel-Agent 能够有效覆盖 Root、Hook、虚拟环境等多类运行时分析技术特征, 具备在复杂指纹环境中精准识别风险痕迹的基础能力.

4.2 不同 LLM 在现实设备指纹数据上的反运行时分析效果对比 (RQ2)

- 实验数据. 为了评估 AIntel-Agent 在现实 Android 设备上的识别性能, 我们从环境设置中展示的 15 台 Android 设备上采集指纹特征, 作为 AIntel-Agent 的输入. 由于现实设备的指纹检测较为复杂, 不同品牌设备对各检测方法的支持情况不同, 因此我们通过人工标注的方式将每个设备的风险指纹特征标记, 通过人工核查的方式统计 AIntel-Agent 的正确识别 (TP)、误报 (FP)、漏报 (FN) 情况.
- 实验设置. 实验所使用的 LLM 均为通过 API 调用, 包括 OpenAI 的 GPT-4o-2024-11-20 和 GPT-oss-120b、Anthropic 的 Claude-sonnet-4-20250514、阿里云百炼的 Qwen3-235b-a22b 以及深度求索的 DeepSeek-reasoner, 本文使用不同的 LLM 分别对 15 台设备的指纹采集结果进行分析.
- 实验结果. 首先关注 AIntel-Agent 选择的方案, 如表 5 所示, 使用 GPT-4o+Claude-sonnet-4 的 AIntel-Agent 在 15 台现实 Android 设备上的平均精确率和召回率达到了 88.5% 和 93.9%, 0.911 的 *F1 score* 显示模型在精确性和全面性之间取得了优秀的平衡. 通过对 workflow 的中间结果进行检查, 误报源于智能体将因定制系统接口问题造成的个别属性在 Java 层和 native 层的检测结果差异作为证据计入评分, 虽然未影响分析结果, 但也被记录为误报. 唯一的漏报来自未将 ro.adb.secure=1 作为运行时分析技术特征报告. 我们将在讨论部分对相关原因进行分析并提出改进思路.

表 5 GPT-4o+Claude-sonnet-4 识别效果

设备	Total	Detected by AIntel-Agent (FP)	Precision (%)	Recall (%)	<i>F1 score</i>
Pixel 5	5	6 (1)	83.3	100.0	0.909
Pixel 6	8	9 (1)	88.9	100.0	0.941
华为P50e	3	3	100.0	100.0	1
华为nova10	4	4 (1)	75.0	75.0	0.75
小米10S	7	7 (1)	85.7	85.7	0.857
红米K50	7	8 (1)	87.5	100.0	0.933
vivo X100S	2	2	100.0	100.0	1
iQOO 10	5	5	100.0	100.0	1
OPPO Reno7	7	6 (1)	83.3	71.4	0.769
一加9R	6	7 (1)	85.7	100.0	0.923
荣耀50	4	4	100.0	100.0	1
荣耀X30	4	4	100.0	100.0	1
努比亚z50	6	6 (1)	83.3	83.3	0.833
红魔6	6	7 (1)	85.7	100.0	0.923
三星s20	8	9 (1)	88.9	100.0	0.941
总体效果	82	87 (10)	88.5	93.9	0.911

注: 括号中的数值为FP

尽管 GPT-4o 与 Claude-sonnet-4 的闭源模型组合表现优异, 但考虑到应用成本、部署灵活性及开源生态的发展, 探究 AIntel-Agent 在其他模型基座上的表现同样至关重要. 为了进一步评估本方法在不同大语言模型下的适应性, 并剖析模型自身的推理与指令遵循能力对复杂反运行时分析任务的具体影响, 我们将实验范围扩展至 GPT-oss、Qwen3 和 DeepSeek-reasoner 这 3 类具有代表性的主流模型. 从表 6 可以看出 AIntel-Agent 使用这 3 种模型时的识别效果与使用 GPT-4o+Claude-sonnet-4 时存在相当差距, 其中 Qwen3 达到 0.64 的 *F1 score*, 在三者中具有显著优势, 但仍存在缺陷. 下面结合具体案例对这 3 种模型的效果做出分析.

以一加 9R 的指纹识别结果为例, Qwen3 能识别出主要运行时分析工具, 但忽视了部分证据并存在误报, 其漏报源于多次忽视 `ro.debuggable=1` 这类不能直接代表风险, 但需要以 Root 或刷入定制 ROM 的方式才可以修改的间接证据, 以及 USB 调试开启等低风险证据. 误报主要来源于将存在特定字符串 (例如“***fake***”) 的属性值判断为存在运行时分析技术特征. GPT-oss 的反运行时分析则围绕一个漏洞 (CVE-2024-53104) 和安全补丁 (2025 年 2 月) 进行, 将所有分析都建立在该基础上, 甚至漏报了发现 `su` 命令这一直接证据. DeepSeek-reasoner 漏报了 BootLoader 解锁、USB 调试等多项证据. GPT-oss 与 DeepSeek-reasoner 均存在大量误报, 会将正常的系统特性 (如高版本 Android 的网络地址匿名化) 错误判定为运行时分析技术特征. 这 3 种模型的主要缺陷在于知识混淆与过度联想, 对网络搜索获取的信息进行了错误分析. 这表明, 即使在设计上添加对获取信息的验证机制和加深认知的循环推理机制, 模型本身的性能问题依然会使任务的执行结果产生差距.

表 6 GPT-oss、Qwen3、DeepSeek-reasoner 识别效果

设备	GPT-oss			Qwen3			DeepSeek-reasoner		
	Precision (%)	Recall (%)	F1 score	Precision (%)	Recall (%)	F1 score	Precision (%)	Recall (%)	F1 score
Pixel 5	50.0	60.0	0.545	50.0	60.0	0.545	28.6	40.0	0.333
Pixel 6	50.0	50.0	0.5	66.7	75.0	0.706	33.3	37.5	0.353
华为P50e	33.3	33.3	0.333	50.0	66.7	0.571	25.0	33.3	0.286
华为nova10	60.0	75.0	0.667	60.0	75.0	0.667	33.3	50.0	0.4
小米10S	42.9	42.9	0.429	71.4	71.4	0.714	33.3	28.6	0.308
红米K50	50.0	42.9	0.462	55.6	71.4	0.625	42.9	42.9	0.429
vivo X100S	50.0	50.0	0.5	50.0	50.0	0.5	50.0	50.0	0.5
iQOO 10	33.3	40.0	0.363	66.7	80.0	0.727	33.3	40.0	0.363
OPPO Reno7	42.9	42.9	0.429	66.7	57.1	0.615	42.9	42.9	0.429
一加9R	42.9	50.0	0.462	66.7	66.7	0.667	33.3	33.3	0.333
荣耀50	66.7	50.0	0.572	60.0	75.0	0.667	40.0	50.0	0.444
荣耀X30	50.0	50.0	0.5	50.0	75.0	0.6	25.0	25.0	0.25
努比亚z50	42.9	50.0	0.462	57.1	66.7	0.615	42.9	50.0	0.462
红魔6	50.0	66.7	0.572	57.1	66.7	0.615	33.3	33.3	0.333
三星s20	40.0	50.0	0.444	62.5	62.5	0.625	44.4	50.0	0.47
总体效果	46.1	50.0	0.48	60.2	68.3	0.64	36.3	40.2	0.382

结论 (RQ2): 大语言模型的性能对分析结果具有显著影响. GPT-4o 与 Claude-sonnet-4 的组合在处理现实设备的复杂指纹时表现最佳 (精确率 88.5%, 召回率 93.9%), 显著优于其他模型, 在面对非结构化数据和长链条推理任务时, 采用具有更强逻辑推理能力的先进模型是保障反运行时分析准确性的关键.

4.3 多智能体系统的性能提升 (RQ3)

• 实验数据. 为了评估 AIntel-Agent 所使用的多智能体系统对性能的影响, 我们构建了作为对比的单智能体工作流程. 两者均使用 GPT-4o+Claude-sonnet-4 模型, 输入第 4.1 节中使用的测试用例与第 4.2 节中现实设备的数据进行测试, 以同样的统计标准计算单智能体的精确率 (Precision)、召回率 (Recall) 和 F1 score.

• 实验设置. 实验所使用的单智能体流程的输入与多智能体系统相同: 一份指纹采集报告和一份人工给出的 Android 设备威胁. 与多智能体系统的区别在于, 单智能体在流程中由一个智能体独立完成整个工作流程. 本文取消了多智能体分工协调的设计, 由该智能体自行检索相关知识, 对指纹采集报告进行分析, 最后输出报告.

• 实验结果. 如表 7 所示, 可以看出, Total 为 84, TP 为 80, FP 为 4, FN 为 4. 计算可得, Precision 为 95.2%, Recall 为 95.2%, F1 score 为 0.952. 对于 12 个测试用例, 单智能体能够识别大部分运行时分析技术特征, 但相较于表 5, 误报频率有所上升, 同时出现漏报情况. 通过对工作流程进行分析, 其性能下降的原因主要源于单智能体在网络搜索与循环推理时的效果与多智能体系统专门执行特定任务的专业智能体存在差异. 如在进行网络搜索时获取的信息全面性不足, 缺失了部分运行时分析工具的信息. 如在 A14-Mix, 本文设置了基于 Xposed 开发的 Hook

框架太极的包名为“me.weishu.exp”, 智能体虽然报告了模拟器和 Root 风险, 但因为动态知识库中未包含太极框架的信息, 而忽视了这一风险.

表 7 单智能体测试用例识别结果

测试用例	Total	Detected by 单智能体 (FP)	测试用例	Total	Detected by AIntel-Agent (FP)
A7-Root	4	4	A13-Virtual	5	5
A8-Virtual	6	7 (1)	A13-Mix	12	10
A9-Hook	5	5	A14-Hook	7	7
A10-Mix	9	10 (1)	A14-Mix	13	12 (1)
A12-Root	4	4	A15-Root	3	3
A12-Mix	8	8	A15-Mix	8	9 (1)

注: 括号中的数值为FP

在现实设备指纹采集结果上的识别效果如表 8 所示, 单智能体在相同数据集上的平均精确率和召回率与 AIntel-Agent 所使用的多智能体系统分别相差 17.2% 和 18.3%, *F1 score* 相差 0.177, 显示这两种设计在精确性和全面性之间存在明显差距. 通过对各步骤的具体输出进行对比, 我们发现差距主要来源于搜索全面性和指纹分析深度的不足. 在单智能体的网络搜索结果中, 常见威胁 (如 Root、Hook 技术) 的结果数量显著高于各威胁检索结果数量的平均水平, 导致动态知识库中对运行时分析技术子类别的描述不够充分, 这可能是导致精确率和召回率下降的主要原因. 在指纹分析方面, 单智能体流程中对指纹采集结果的疑点识别和知识获取行为的数量相比多智能体系统显著减少, 使部分无害特征未在分析后被排除. 以 vivo X100S 为例, vivo 在设备型号上同时使用了 PD2309 和 PD2309M 两种命名, 这在多智能体系统中被成功排除, 而单智能体将其作为运行时分析技术特征报告. 这也印证了一些关于智能体的研究观点: 明确的计划可以使智能体更高效地完成任务^[13].

表 8 多智能体系统与单智能体识别效果对比

设备	多智能体			单智能体		
	Precision (%)	Recall (%)	<i>F1 score</i>	Precision (%)	Recall (%)	<i>F1 score</i>
Pixel 5	83.3	100.0	0.909	80.0	80.0	0.8
Pixel 6	88.9	100.0	0.941	75.0	75.0	0.75
华为P50e	100.0	100.0	1	66.7	66.7	0.667
华为nova10	75.0	75.0	0.75	60.0	75.0	0.667
小米10S	85.7	85.7	0.857	71.4	71.4	0.714
红米K50	87.5	100.0	0.933	83.3	71.4	0.769
vivo X100S	100.0	100.0	1	50.0	100.0	0.667
iQOO 10	100.0	100.0	1	80.0	80.0	0.8
OPPO Reno7	83.3	71.4	0.769	83.3	71.4	0.769
一加9R	85.7	100.0	0.923	80.0	66.7	0.727
荣耀50	100.0	100.0	1	60.0	75.0	0.667
荣耀X30	100.0	100.0	1	80.0	100.0	0.889
努比亚z50	83.3	83.3	0.833	71.4	83.3	0.769
红魔6	85.7	100.0	0.923	66.7	66.7	0.667
三星s20	88.9	100.0	0.941	60.0	75.0	0.667
总体效果	88.5	93.9	0.911	71.3	75.6	0.734

结论 (RQ3): 多智能体协作架构相较于单智能体模式在精确率和召回率上分别提升了 17.2% 和 18.3%. 这证明了将复杂的反运行时分析任务分解为规划、搜索、分析等子任务并分配给专业智能体执行, 能够有效解决单一智能体在能力上的局限, 显著提升分析效果.

4.4 不同智能体核心模块的消融实验 (RQ4)

- 实验数据. 为了验证我们设计的循环推理机制对 AIntel-Agent 性能的影响, 我们使用第 4.1 与 4.2 节中的数

据作为输入, 通过相同的标准对消融循环推理模块前后 AIntel-Agent 的正确识别 (TP)、误报 (FP)、漏报 (FN) 情况进行统计.

● 实验设置. 由于各智能体均在工作流程中起到关键作用, 直接移除会使 AIntel-Agent 整体的功能退化严重, 作为替代我们分别移除了不同智能体的关键模块, 通过对比移除模块前后的表现, 我们可以清晰地评估该模块对提升分析质量的实际贡献. 在威胁研究规划员中, 本文取消了接收人工给出的搜索基准, 由智能体自行进行搜索与总结; 在风险知识检索员中, 我们移除了搜索后对 URL 验证的模块, 将搜索结果直接整理为风险标签; 在指纹风险分析员中, 我们移除了循环推理模块, 改为由智能体直接将指纹采集结果与风险标签相对应; 在智能体协调员中, 我们取消了其规划任务、调度智能体的功能, 只进行数据分发的的工作.

● 实验结果. 如表 9 所示, 在消融智能体功能模块后, AIntel-Agent 在反运行时分析任务上的性能均有所下降.

表 9 消融实验前后性能对比 (%)

设备	威胁研究规划员		风险知识检索员		指纹风险分析员		智能体协调员	
	Precision	Recall	Precision	Recall	Precision	Recall	Precision	Recall
Pixel 5	80.0	80.0	80.0	80.0	71.4	100.0	83.3	100.0
Pixel 6	83.3	62.5	70.0	87.5	60.0	75.0	77.8	87.5
华为P50e	75.0	100.0	100.0	100.0	66.7	66.7	50.0	66.7
华为nova10	75.0	75.0	80.0	100.0	66.7	100.0	80.0	100.0
小米10S	71.4	71.4	75.0	85.7	60.0	85.7	66.7	85.7
红米K50	66.7	57.1	85.7	85.7	63.6	100.0	77.8	100.0
vivo X100S	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
iQOO 10	60.0	60.0	66.7	80.0	71.4	100.0	80.0	80.0
OPPO Reno7	71.4	71.4	71.4	71.4	66.7	85.7	85.7	85.7
一加9R	71.4	83.3	85.7	100.0	54.5	100.0	71.4	83.3
荣耀50	75.0	75.0	100.0	100.0	66.7	100.0	60.0	75.0
荣耀X30	100.0	100.0	80.0	100.0	60.0	75.0	80.0	100.0
努比亚z50	66.7	66.7	66.7	66.7	62.5	83.3	71.4	83.3
红魔6	71.4	83.3	83.3	83.3	62.5	83.3	62.5	83.3
三星s20	85.7	75.0	77.8	87.5	58.3	87.5	70.0	87.5
总体效果	75.3	74.4	78.9	86.6	64.0	89.0	74.2	87.8

在取消威胁研究规划员接收的搜索基准后, AIntel-Agent 的精确率由 88.5% 下降至 75.3%, 召回率由 93.9% 下降至 74.4%. 检查智能体的输出后, 发现智能体对不同运行时分析技术的搜索关键词数量明显不平衡, 在测试中, Root 与 Hook 相关的关键词占比超过 60%, 对比其他运行时分析技术的搜索深度相较未消融时明显下降, 这可能是系统性能下降的主要原因.

在移除风险知识检索员的知识验证模块后, AIntel-Agent 的精确率由 88.5% 下降至 78.9%, 召回率由 93.9% 下降至 86.6%, 整体性能下降幅度较小. 由于目前智能体网络搜索的局限性, 在每次测试的 125 个搜索结果中, 有约 40 个返回无效 URL. 目前最先进的 LLM 可以凭借自身性能容忍部分错误输入, 但知识验证模块将这部分无法验证来源的信息排除仍可以使性能得到一定提升.

在移除指纹风险分析员的循环推理模块后, AIntel-Agent 的精确率由 88.5% 下降至 64.0%, 召回率由 93.9% 下降至 89.0%, 精确率显著下降. 以一加 9R 的测试为例, 误报的增加主要来自智能体凭空捏造了一个“Android 14 存在关键漏洞”的风险, 并将多项正常属性 (如 API 级别) 识别为风险特征. 实验表明循环推理机制不仅能够提升 AIntel-Agent 的分析深度, 更是约束 LLM 基于证据链条进行推理的关键组件, 在没有约束的情况下, 即使是最先进的 LLM 也容易在复杂的分析任务中产生没有根据的结论.

在取消智能体协调员规划任务、调度智能体的能力后, AIntel-Agent 的精确率由 88.5% 下降至 74.2%, 召回率由 93.9% 下降至 87.8%. 由于各智能体在设计中具有独立完成任务分析、工具调用和结果思考的能力, 在缺少智

能体协调员的调度后依然能完成主要工作,但在缺乏总体规划与任务分配的情况下, AIntel-Agent 的精确率和召回率都受到影响。

综合来看,实验结果表明,各智能体设计中的模块都能有效提升 AIntel-Agent 的性能,取消任何模块都会影响 AIntel-Agent 执行运行时分析任务的效果。

结论 (RQ4): AIntel-Agent 中的每个智能体都对最终性能做出了不可或缺的贡献。移除任一智能体的关键模块都会使 AIntel-Agent 的分析结果劣化,这证明了 AIntel-Agent 的动态协作机制能够引导智能体捕捉更深层的隐藏特征,并主动剔除误报内容,验证了智能体协作架构设计的合理性与必要性。

5 讨 论

5.1 结果思考

本文实验系统性地验证了 AIntel-Agent 作为一个基于威胁情报驱动的多智能体 Android 运行时分析方法的有效性与潜力。在合成数据集上的测试证明了该方法能够精准映射运行时分析技术特征;在现实设备环境下的评估显示 AIntel-Agent 的最终性能与大语言模型的性能相关,但 AIntel-Agent 的设计是实现高效、准确分析的必要前提。相较于单智能体, AIntel-Agent 的多智能体架构通过明确的角色分工,有效解决了单一智能体在知识探索时面临的“广度”与“深度”难以兼顾的困境,多智能体架构通过角色分工和明确的计划引导,避免智能体在分析中过早收敛于常见威胁而忽略了更隐蔽的运行时分析痕迹。同时,“先计划、再行动、后反思”的循环推理机制在约束 LLM 分析过程中起到了重要作用,有效提升了分析的深度与可靠性。循环推理强制模型从孤立、发散的特征开始联想,结合不同威胁与运行时分析技术的指纹特征构建收敛性的、相互印证的证据链,引导智能体将多个分散的指纹属性归因到一个高阶结论,再逆向思考是否还有未识别的运行时分析特征。

Android 平台上对于反运行时分析已有多项分析研究,然而,由于 AIntel-Agent 与已有研究在实现逻辑与检测对象方面存在显著差异,难以进行直接的对比实验。但与相关研究相比, AIntel-Agent 的设计针对性地解决了现有方法的两大核心局限。一方面,基于静态知识库的反运行时分析方法受限于其“知识滞后”的固有缺陷,难以应对新兴攻击和对抗性样本。 AIntel-Agent 通过从开放互联网实时构建动态威胁知识库,克服了对静态数据库的依赖,使其具备跟进最新安全威胁的能力。另一方面,尽管已有研究尝试应用大语言模型进行安全分析,但单一智能体在面对 Android 反运行时分析这类长链条、知识密集的复杂任务时,存在规划与推理能力不足、信息获取不可靠的问题。 AIntel-Agent 通过多智能体协作架构将复杂任务分解,并通过循环推理机制模拟专家认知过程,系统性地解决了单一智能体分析深度不足和结果不稳定的难题。

AIntel-Agent 通过将人类专家的分析流程形式化并注入多智能体系统,使 Android 反运行时分析不再局限于匹配已知的风险特征,而是能够像人类专家一样,基于动态获取的情报对未知和隐蔽的威胁进行逻辑推理。这不仅为应对快速演变的 Android 安全威胁提供了有效途径,也为如何将通用大语言模型的能力可靠、可控地应用于其他专业化、高复杂性的安全领域,提供了一个具有参考价值的实现方式。

5.2 有效性威胁

尽管本文提出的 AIntel-Agent 通过实验对其性能进行了多角度验证,但仍存在一些可能影响研究结果泛化能力与解释力的有效性威胁。

1) 不确定性信息处理偏差。在部分测试用例中,当面对低版本 Android 系统的性能特征或应用权限不足导致的“unknown”状态时,智能体倾向于将其判断为风险。同样,偶发的误报、漏报案例也表明,在复杂的指纹组合中,智能体对低风险或辅助性证据的权重判断仍有待优化。为此,本文设计了形式化的概率评分体系,要求智能体不得将“unknown”状态作为唯一判断依据,只有在已获取明确风险证据的前提下才可将其作为辅助证据。

2) LLM 脆弱性。 AIntel-Agent 的实现依赖于大语言模型,这也使其继承了 LLM 固有的脆弱性,例如通过在输入中隐藏特定提示词引导 LLM 输出的方式影响 AIntel-Agent 结果。一个典型的例子是,模型可能被指纹中出现的“fake”等强引导性语义词汇误导,做出与技术事实不符的判断,这可能对 AIntel-Agent 的有效性造成威胁。

3) 知识来源的可靠性. AIntel-Agent 通过动态构建知识库摆脱了静态数据库的“知识滞后”问题,但也因此引入了对开放互联网信息质量的依赖. 尽管本文设计了 URL 验证机制,但这无法从根本上解决信息源本身可能被错误、过时甚至恶意内容污染的问题. 为此,我们引入了多源验证与来源权重机制,智能体在生成风险标签时会综合多个来源的信息验证其对某一风险的描述是否一致,并会在判断时增加权威数据来源的权重.

5.3 未来工作

1) 扩展测试基准与模型库. 本文实验使用的 Android 设备较少,在性能对比中涵盖的大模型数量有限. 这在一定程度上限制了实验结论的泛化能力. 不同品牌、不同 Android 定制系统的设备可能存在独特的系统行为和指纹特征,而不同的 LLM 也存在各自的优势,更大规模的测试集将有助于更全面地评估 AIntel-Agent 的鲁棒性和适应性. 未来我们将纳入更多品牌、更多 Android 版本的测试设备,构建一个更具多样性和挑战性的基准数据集. 同时,我们也将引入更多 LLM 进行对比测试,以期建立一个更全面的 LLM 在 Android 安全领域的能力图谱.

2) 调整智能体对风险特征的判断. 目前智能体在对设备指纹进行分析时,可能会因过低 Android 版本、特定的电池状态出现误判,将真实设备误报为虚拟设备. 我们已引入了形式化的概率评分体系要求智能体只能在获取到足够数量或强度的证据时才可确认风险的存在. 未来,我们会尝试通过人为划分风险特征优先级等方式在不忽视风险的前提下,降低此类误报发生的可能性. 同时,也会持续对我们开发的设备指纹采集 App 进行优化,确保其返回的指纹数据清晰且意义明确,不会使智能体难以理解或做出误判.

3) LLM 的混合优化策略. 针对当前方案依赖昂贵闭源 API 的问题,我们尝试探索一种混合模型策略. 例如,可以尝试在知识库构建的初步搜索、内容总结等流程中,使用性能相近的低成本模型或经过微调的开源模型,而在核心的循环推理与风险归因阶段,使用高性能或具有特殊优势的模型. 通过这种方式,在保证核心分析性能的前提下,可以寻找更多可用的 LLM 组合方案,提高 AIntel-Agent 的泛用性.

4) 知识来源的多元化与可靠性验证. 目前基于网络搜索构建的动态知识库能够满足反运行时分析任务的需求,但信息来源的可靠性是当前基于网络搜索的智能体面临的共性问题,尽管我们已引入了多源验证与来源权重,但随着对抗的升级,我们将致力于建立更深层的对抗性情报防御体系,例如引入自动化的网站信誉评价模型,整合权威威胁情报源验证社区数据真伪,降低系统在面对情报投毒和虚假隐藏技巧注入时的受攻击面,从而保障智能体推理逻辑的安全性和鲁棒性.

5) 端侧大模型的应用可行性与隐私保护优化. 考虑到实际检测中用户数据的隐私保护问题,我们将尝试通过设备端本地化的 LLM 进行分析. 探索知识蒸馏技术,采用教师-学生蒸馏范式,以 LLM 作为教师模型,生成高质量推理链、证据链和结构化决策,再将其迁移到轻量级学生模型中,使学生模型在保持推理效率的同时获得强大的分析能力,满足隐私性与推理能力之间的平衡需求. 通过利用日益强大的移动端计算框架,在有限的端侧资源下实现数据的本地分析,解决敏感指纹数据上传带来的隐私泄露与合规风险,并降低系统的长期运行成本.

6 总 结

AIntel-Agent 旨在为 Android 设备提供一个自动化、智能化的反运行时分析方法,以应对持续演进的复杂安全威胁. 为此,本文系统性地克服了智能体能力局限、网络搜索的可靠性和指纹分析深度不足等核心挑战. 实验评估表明, AIntel-Agent 能够准确、高效地识别多层次运行时分析技术特征,在关键的反运行时分析任务中表现出卓越的精确率与召回率,并能生成高质量、可信的分析报告. 随着 Android 生态的持续演进与大语言模型技术的不断革新, AIntel-Agent 将在更广泛的设备与威胁场景中优化和迭代,并探索端侧模型部署等前沿方向,以期在保障用户隐私的同时,长期为 Android 生态安全提供智能化的分析能力.

References

- [1] StatCounter. <https://gs.statcounter.com/os-market-share/mobile/worldwide>
- [2] Rathore H, Sahay SK, Nikam P, Sewak M. Robust Android malware detection system against adversarial attacks using Q-learning. Information Systems Frontiers, 2021, 23(4): 867–882. [doi: 10.1007/s10796-020-10083-8]

- [3] Tam K, Feizollah A, Anuar NB, Salleh R, Cavallaro L. The evolution of Android malware and Android analysis techniques. *ACM Computing Surveys (CSUR)*, 2017, 49(4): 76. [doi: [10.1145/3017427](https://doi.org/10.1145/3017427)]
- [4] Suo DW, Xue L, Yu L, Tan RZ, Huang WH, Sun GZ. ARAP: Demystifying anti runtime analysis code in Android apps. *IEEE Trans. on Software Engineering*, 2025, 51(10): 2787–2803. [doi: [10.1109/TSE.2025.3596016](https://doi.org/10.1109/TSE.2025.3596016)]
- [5] Nguyen-Vu L, Chau NT, Kang S, Jung S. Android rooting: An arms race between evasion and detection. *Security and Communication Networks*, 2017, 2017: 4121765. [doi: [10.1155/2017/4121765](https://doi.org/10.1155/2017/4121765)]
- [6] Sihag V, Vardhan M, Singh P. A survey of Android application and malware hardening. *Computer Science Review*, 2021, 39: 100365. [doi: [10.1016/j.cosrev.2021.100365](https://doi.org/10.1016/j.cosrev.2021.100365)]
- [7] Apostolopoulos T, Katos V, Choo KKR, Patsakis C. Resurrecting anti-virtualization and anti-debugging: Unhooking your hooks. *Future Generation Computer Systems*, 2021, 116: 393–405. [doi: [10.1016/j.future.2020.11.004](https://doi.org/10.1016/j.future.2020.11.004)]
- [8] Alamro H, Mtouaa W, Aljameel S, Salama AS, Hamza MA, Othman AY. Automated Android malware detection using optimal ensemble learning approach for cybersecurity. *IEEE Access*, 2023, 11: 72509–72517. [doi: [10.1109/ACCESS.2023.3294263](https://doi.org/10.1109/ACCESS.2023.3294263)]
- [9] Merlo A, Georgiu GC. RiskInDroid: Machine learning-based risk analysis on Android. In: *Proc. of the 32nd IFIP Int'l Conf. ICT Systems Security and Privacy Protection*. Rome: Springer, 2017. 538–552. [doi: [10.1007/978-3-319-58469-0_36](https://doi.org/10.1007/978-3-319-58469-0_36)]
- [10] Moon IT, Shamsuzzaman M, Mridha MER, Rahaman ASMM. Towards the advancement of cashless transaction: A security analysis of electronic payment systems. *Journal of Computer and Communications*, 2022, 10(7): 103–129. [doi: [10.4236/jcc.2022.107007](https://doi.org/10.4236/jcc.2022.107007)]
- [11] Kondracki B, Azad BA, Miramirkhani N, Nikiforakis N. The droid is in the details: Environment-aware evasion of Android sandboxes. In: *Proc. of the 29th Annual Network and Distributed System Security Symp.* San Diego: The Internet Society, 2022.
- [12] Hong YF, Hu YJ, Lai CM, Wu SF, Neamtui I, McDaniel P, Yu P, Çam H, Ahn GJ. Defining and detecting environment discrimination in Android apps. In: *Proc. of the 13th Int'l Conf. on Security and Privacy in Communication Systems*. Niagara Falls: Springer, 2018. 510–529. [doi: [10.1007/978-3-319-78813-5_26](https://doi.org/10.1007/978-3-319-78813-5_26)]
- [13] Xi ZH, Chen WX, Guo X, He W, Ding YW, Hong BY, Zhang M, Wang JZ, Jin SJ, Zhou EY, Zheng R, Fan XR, Wang X, Xiong LM, Zhou YH, Wang WR, Jiang CH, Zou YC, Liu XY, Yin ZY, Dou SH, Weng RX, Qin WJ, Zheng YY, Qiu XP, Huang XJ, Zhang Q, Gui T. The rise and potential of large language model based Agents: A survey. *Science China Information Sciences*, 2025, 68(2): 121101. [doi: [10.1007/s11432-024-4222-0](https://doi.org/10.1007/s11432-024-4222-0)]
- [14] Shao YJ, Jiang YC, Kanell T, Xu P, Khattab O, Lam M. Assisting in writing wikipedia-like articles from scratch with large language models. In: *Proc. of the 2024 Conf. of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Mexico City: ACL, 2024. 6252–6278. [doi: [10.18653/v1/2024.naacl-long.347](https://doi.org/10.18653/v1/2024.naacl-long.347)]
- [15] Tang JB, Fan TY, Huang C. AutoAgent: A fully-automated and zero-code framework for LLM agents. *arXiv:2502.05957*, 2025.
- [16] Hu MK, Zhou YH, Fan WD, Nie YZ, Xia BW, Sun T, Ye ZY, Jin ZX, Li YR, Chen QG, Zhang ZY, Wang YF, Ye QH, Ghanem B, Luo P, Li GH. OWL: Optimized workforce learning for general multi-agent assistance in real-world task automation. *arXiv:2505.23885*, 2025.
- [17] Pan RW, Zhang HY, Jiang ZH, Hou R. AgentDroid: A multi-agent tool for detecting fraudulent Android applications. In: *Proc. of the 40th IEEE/ACM Int'l Conf. on Automated Software Engineering*. Seoul: IEEE, 2025. 4009–4012. [doi: [10.1109/ASE63991.2025.00362](https://doi.org/10.1109/ASE63991.2025.00362)]
- [18] Mary SS. Common security vulnerabilities in Android Apps: A comprehensive guide. *Int'l Journal for Multidisciplinary Research*, 2024, 6(6): 1–15. [doi: [10.36948/ijfmr.2024.v06i06.32931](https://doi.org/10.36948/ijfmr.2024.v06i06.32931)]
- [19] Liu YC. Retrieval-augmented generation: Methods, applications and challenges. *Applied and Computational Engineering*, 2025, 142: 99–108. [doi: [10.54254/2755-2721/2025.KL22312](https://doi.org/10.54254/2755-2721/2025.KL22312)]
- [20] Shi WX, Tan HC, Kuang CQ, Li XG, Ren XZ, Zhang C, Chen HT, Wang YS, Shang LF, Yu F, Wang YH. Pangu DeepDiver: Adaptive search intensity scaling via open-Web reinforcement learning. *arXiv:2505.24332*, 2025.
- [21] Xing MZ, Zhang RK, Xue H, Chen Q, Yang F, Xiao Z. Understanding the weakness of large language model agents within a complex Android environment. In: *Proc. of the 30th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining*. Barcelona: ACM, 2024. 6061–6072. [doi: [10.1145/3637528.3671650](https://doi.org/10.1145/3637528.3671650)]
- [22] Zhang LY, Liu CW, Wu JH, Zhang SY, Liu CY, Xu ZZ, Chen S, Liu Y. Drop the golden apples: Identifying third-party reuse by DB-less software composition analysis. In: *Proc. of the 33rd ACM Int'l Conf. on the Foundations of Software Engineering*. Clarion Hotel: ACM, 2025. 691–695. [doi: [10.1145/3696630.3728525](https://doi.org/10.1145/3696630.3728525)]
- [23] Baresi L, Caushi K. IDEA: Runtime collection of Android data. In: *Proc. of the 2021 IEEE Int'l Symp. on Software Reliability Engineering Workshops*. Wuhan: IEEE, 2021. 137–143. [doi: [10.1109/ISSREW53611.2021.00055](https://doi.org/10.1109/ISSREW53611.2021.00055)]
- [24] Han WL, Cao C, Chen H, Li D, Fang ZR, Xu WY, Wang XS. SenDroid: Auditing sensor access in Android system-wide. *IEEE Trans. on Dependable and Secure Computing*, 2020, 17(2): 407–421. [doi: [10.1109/TDSC.2017.2768536](https://doi.org/10.1109/TDSC.2017.2768536)]
- [25] Torres CF, Jonker H. Investigating fingerprinters and fingerprinting-alike behaviour of Android applications. In: *Proc. of the 23rd*

- European Symp. on Research in Computer Security. Barcelona: Springer, 2018. 60–80. [doi: [10.1007/978-3-319-98989-1_4](https://doi.org/10.1007/978-3-319-98989-1_4)]
- [26] Guo SY, Deng C, Wen Y, Chen HC, Chang Y, Wang J. DS-Agent: Automated data science by empowering large language models with case-based reasoning. In: Proc. of the 41st Int'l Conf. on Machine Learning. Vienna: PMLR, 2024. 16813–16848.
- [27] Huang YX, Chen YH, Zhang HZ, Li K, Zhou HC, Fang M, Yang LY, Li XG, Shang LF, Xu SC, Hao JY, Shao K, Wang J. Deep research agents: A systematic examination and roadmap. arXiv:2506.18096, 2025.
- [28] LSPosed. 2021. <https://github.com/LSPosed/LSPosed>
- [29] APatch. 2023. <https://github.com/bmax121/APatch>
- [30] KernelSU. 2023. <https://github.com/tiann/KernelSU>
- [31] Coolapk (in Chinese). <https://www.coolapk.com/>
- [32] Reddit. <https://www.reddit.com/login/?rdt=45187>
- [33] MagiskDetection/Momo-v4.4.1.apk. 2022. <https://github.com/apkunpacker/MagiskDetection/blob/main/Momo-v4.4.1.apk>
- [34] Alibaba Cloud security vulnerability DB (in Chinese). <https://avd.aliyun.com/>
- [35] Android security bulletins (in Chinese). <https://source.android.google.cn/docs/security/bulletin>
- [36] OWASP mobile application security. <https://mas.owasp.org/>
- [37] Li XX, Jin JJ, Dong GT, Qian HJ, Wu YK, Wen JR, Zhu YT, Dou ZC. WebThinker: Empowering large reasoning models with deep research capability. arXiv:2504.21776, 2025.
- [38] Jin BW, Zeng HS, Yue ZR, Yoon J, Arik S, Wang D, Zamani H, Han JW. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. arXiv:2503.09516, 2025.
- [39] Han RJ, Chen YF, CuiZhu Z, Miculicich L, Sun G, Bi YJ, Wen WM, Wan H, Wen CF, Maître S, Lee G, Tirumalashetty V, Xue E, Zhang ZZ, Haykal S, Gokturk B, Pfister T, Lee CY. Deep researcher with test-time diffusion. arXiv:2507.16075, 2025.

附中文参考文献

- [31] 酷安. <https://www.coolapk.com/>
- [34] 阿里云漏洞库. <https://avd.aliyun.com/>
- [35] Android 平台专用公告. <https://source.android.google.cn/docs/security/bulletin>

作者简介

严欣苒, 硕士生, 主要研究领域为软件供应链安全, Android 应用程序分析.

张向宇, 博士生, CCF 学生会员, 主要研究领域为移动安全, Android 应用程序分析, 智能体安全.

范玲玲, 博士, 副教授, 博士生导师, CCF 专业会员, 主要研究领域为软件测试, 软件安全, 软件供应链安全.

苏煜程, 本科, 工程师, 主要研究领域为移动安全, 应用程序分析.

陈森, 博士, 教授, 博士生导师, CCF 高级会员, 主要研究领域为软件安全, 软件供应链安全.

刘哲理, 博士, 教授, 博士生导师, CCF 专业会员, 主要研究领域为数据安全, 人工智能安全.

操晓春, 博士, 教授, 博士生导师, CCF 专业会员, 主要研究领域为人工智能安全, 网络空间内容安全.